SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

FACULTY OF ENGINEERING AND TECHNOLOGY

VADAPALANI – 600 026



E-COMMERCE ENTERPRISE APPLICATION USING SPRING BOOT, HIBERNATE AND NEON SERVERLESS POSTGRESQL

A PROJECT REPORT

Submitted by

PRANESH M S [RA2511003040005]

AVINASH A S [RA2511003040074]

KABIL SAMY K M [RA2511003040084]

DINESH KUMAR [RA2511003040106]

ACADEMIC YEAR

2025-2026



SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
Deemed to be University u/s 3 of UGC Act, 1956
VADAPALANI

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

Certified that 21CSC203P Advanced Programming Practice titled **"E-COMMERCE ENTERPRISE APPLICATION USING SPRING BOOT, HIBERNATE AND NEON SERVERLESS POSTGRESQL"** is a bonafide work of **"PRANESH M S [RA2511003040005], AVINASH A S [RA2511003040074], KABIL SAMY K M [RA2511003040084], DINESH KUMAR [RA2511003040106]"** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported here in does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE **SIGNATURE**

STAFF IN-CHARGE **HEAD OF THE DEPARTMENT**

TABLE OF CONTENTS

CONTENT	PAGE NO
1. INTRODUCTION 1.1 PROJECT AIM AND OBJECTIVES 1.2 BACKGROUND OF PROJECT	**2**
2. SYSTEM ANALYSIS 2.1 SOFTWARE REQUIREMENT SPECIFICATION 2.1.1 GENERAL DESCRIPTION 2.1.2 SYSTEM OBJECTIVES 2.1.3 SYSTEM REQUIREMENTS 2.2 SOFTWARE TOOLS USED	**4**
3. SYSTEM DESIGN 3.1 DATABASE SCREENSHOTS 3.2 DATA FLOW DIAGRAMS & USE CASE	**8**
4. SYSTEM IMPLEMENTATION 4.1 SCREEN SHOTS OF OUTPUT 4.2 SCREENSHOTS OF CODING PART	**13**
5. SYSTEM TESTING 5.1 FUNCTIONALITY TESTING 5.2 EXCEPTION TESTING WITHIN FUNCTIONAL TESTING 5.3 EXAMPLES (TEST CASES MATRIX)	**26**
6. CONCLUSION	**31**
7. REFERENCES	**32**

ABSTRACT:

The E-Commerce Enterprise Web Application is a production-oriented, scalable digital shopping platform engineered using Spring Boot, Hibernate ORM, and Neon Serverless PostgreSQL. Designed to eliminate the limitations of conventional offline stores and fragmented inventory systems, the application provides an end-to-end e-commerce workflow catering to both public consumers and administrative personnel.

The platform features an architectural separation of concerns leveraging the Model-View-Controller (MVC) paradigm. The presentation layer utilizes high-performance JavaServer Pages (JSP) paired with Bootstrap to render responsive, cross-platform user interfaces. The business logic is orchestrated through Spring Boot controllers and transaction-managed service layers, while the persistence tier utilizes a customized Hibernate SessionFactory connected securely to a serverless, cloud-hosted PostgreSQL database on Neon via SSL.

Role-Based Access Control (RBAC) is implemented via Spring Security, providing strictly isolated operational boundaries between customer activities (user registration, catalog browsing, and cart checkouts) and administrative governance (category orchestration, real-time product updates, and customer directory administration).

By moving the database to Neon's cloud-native serverless architecture, the system guarantees elastic compute scaling, automated connection pooling, and continuous high availability. The comprehensive project report demonstrates the complete system engineering lifecycle, including requirement specifications, relational database design, implementation snapshots, testing methodologies, and deployment configurations.

CHAPTER 1
INTRODUCTION

This chapter presents an introduction to the E-Commerce Enterprise Web Application project. It discusses the system's aim, objectives, and background, along with the operational environment in which the enterprise application operates.

1.1 PROJECT AIM AND OBJECTIVES

Aim: The primary aim of the E-Commerce Enterprise Web Application project is to develop a reliable, highly available, and user-friendly digital marketplace platform that facilitates seamless product discovery, cart transactions, and inventory administration. The system aims to leverage modern enterprise Java architectures and serverless cloud databases to provide responsive and secure electronic commerce operations.

Objectives:

- To design an interactive, responsive web storefront allowing customers to browse categorized products, inspect prices, and manage shopping items with ease.
- To implement robust user authentication and role-based access control (RBAC), securing sensitive administrative modules from unauthorized public access.
- To build a dedicated administrative dashboard for managing product catalogs, updating category taxonomies, and reviewing registered consumer profiles.
- To configure a custom Hibernate 5 SessionFactory data access tier for efficient ORM object mapping, transactional integrity, and optimized query execution.
- To deploy and integrate the application with Neon Serverless PostgreSQL, achieving secure cloud-native data persistence with SSL encryption.

1.2 BACKGROUND OF PROJECT

- In today's fast-paced digital economy, electronic commerce has evolved into an essential foundation for modern commerce and retail. Consumers increasingly expect instantaneous access to product inventories, transparent pricing, and dependable online shopping experiences accessible across desktop and mobile devices. Traditional brick-and-mortar retail and legacy desktop inventory systems suffer from geographic constraints, high operational overhead, error-prone manual reconciliation, and a lack of real-time inventory synchronization.
- With the rapid advancement of enterprise web frameworks and cloud infrastructure, building robust, distributed, server-rendered applications has become the industry standard for commercial platforms. Spring Boot provides an enterprise-grade ecosystem that simplifies microservice and web application bootstrapping, dependency injection, and security enforcement.
- Simultaneously, traditional database hosting has encountered significant hurdles with static resource provisioning, manual database scaling, and local maintenance overhead. The integration of modern cloud serverless databases, exemplified by Neon Serverless PostgreSQL, resolves these bottlenecks by offering on-demand auto-scaling, pooled connection handling, and robust SSL-encrypted cloud storage.
- The E-Commerce Enterprise Web Application project was conceived to address these technical challenges by integrating Spring Boot's layered MVC framework, Hibernate's reliable SessionFactory persistence, and Neon's cloud-native database to deliver a fully functional, enterprise-grade e-commerce system.

CHAPTER 2
SYSTEM ANALYSIS

2.1 SOFTWARE REQUIREMENT SPECIFICATION

2.1.1 GENERAL DESCRIPTION:

- The E-Commerce Enterprise Web Application is a web-based full-stack shopping and management platform developed to streamline online retail workflows. It enables customers to register secure accounts, explore categorized goods (Fruits, Bakery, Dairy, Meat, Drinks, etc.), inspect product descriptions, and perform transactional cart purchases in real time. The system eliminates manual inventory logging by providing an automated administrative hub.
- Administrators are empowered with comprehensive governance capabilities, allowing them to create, modify, and delete product listings, adjust inventory pricing and stock quantities, create new product categories, and audit registered customer accounts. Visual alerts and dynamic status indicators inform administrators of catalog updates and operational actions.
- The system operates seamlessly across modern web browsers (Chrome, Edge, Firefox, Safari) and leverages an embedded Tomcat 9 server for request dispatching. All persistent records—including user credentials, category classifications, product inventories, and cart items—are housed within a cloud-hosted Neon Serverless PostgreSQL database.
- The architecture emphasizes security, modularity, and operational reliability, utilizing Spring Security for credential hashing and role verification, ensuring strict data integrity and protection against common web vulnerabilities.

2.1.2 SYSTEM OBJECTIVES

The central objective of the E-Commerce Enterprise Web Application is to provide a unified, performant, and secure platform that facilitates commercial interactions between consumers and catalog administrators. The system is designed to minimize operational bottlenecks, ensure data integrity, and guarantee uninterrupted catalog availability through cloud-native hosting.

Specific Objectives:

- **1. Automated Product Catalog Management:**
  Simplify the ingestion, updating, and categorization of commercial products through dynamic administrative interfaces, eliminating manual bookkeeping errors.
- **2. Role-Based Identity and Access Management:**
  Enforce strict role separation between standard customers (ROLE_NORMAL) and system supervisors (ROLE_ADMIN) to prevent privilege escalation and secure administrative endpoints.
- **3. Transactional Shopping Cart Operations:**
  Provide dependable shopping cart aggregation with persistent state tracking, allowing users to select items, inspect order totals, and initiate checkout workflows.
- **4. Serverless Cloud Database Resilience:**
  Guarantee high availability, data durability, and automated compute scaling by migrating the persistence tier to Neon Serverless PostgreSQL over secure SSL connections.
- **5. Intuitive User Experience:**
  Develop clean, responsive JSP interfaces styled with modern CSS and Bootstrap that ensure accessible shopping and administrative workflows across all display form factors.

2.1.3 SYSTEM REQUIREMENTS:

A. Hardware Requirements:

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher (Quad-core 2.0 GHz recommended for application hosting).
- **RAM:** Minimum 4 GB RAM (8 GB recommended for concurrent JVM and server execution).
- **Storage:** Minimum 500 MB free disk space for application packages, compiled artifacts, and Maven dependencies.
- **Network:** High-speed broadband internet connectivity for cloud database communication with Neon DB.
- **Display:** Standard HD display (1366 x 768 or higher) for administrative portal monitoring and user browsing.

B. Software Requirements:

- **Operating System:** Windows 10/11, Linux (Ubuntu/Debian), or macOS.
- **Runtime Environment:** Java Development Kit (JDK) 17 LTS (Microsoft Build of OpenJDK 17).
- **Framework:** Spring Boot Framework version 2.6.4 (Servlet stack, Spring MVC, Spring Security).
- **Persistence Framework:** Hibernate ORM version 5.6.5 Final with custom SessionFactory configuration.
- **Database:** Neon Serverless PostgreSQL (PostgreSQL 15 engine with SSL connectivity).
- **Build Tool:** Apache Maven 3.8.6 with Maven Wrapper integration.
- **Web Server:** Embedded Apache Tomcat 9.0.58.
- **Web Browser:** Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari.

C. User Requirements:

- End users must have access to a web browser and an active internet connection.
- Users must be able to register a unique profile and log in securely.
- Customers require intuitive navigation to view items, filter categories, and manage cart totals.
- Administrators require secure access credentials to manage products, categories, and customer records.

2.2 SOFTWARE TOOLS USED :

- **Spring Boot 2.6.4:** Core Java framework used for dependency injection, component scanning, auto-configuration, and orchestrating the MVC web tier.
- **Hibernate ORM 5.6.5:** Object-Relational Mapping (ORM) framework providing automated schema synchronization, entity mapping, and SessionFactory transaction handling.
- **Neon Serverless PostgreSQL:** Cloud-native database provider offering on-demand serverless PostgreSQL storage, pooled connectivity, and automated scaling.
- **Java 17 LTS:** Modern, stable Long Term Support Java programming language runtime powering backend compilation and execution.
- **JavaServer Pages (JSP) & JSTL:** Server-side template rendering technology used to dynamically generate responsive HTML views for customers and administrators.
- **Spring Security:** Robust security framework providing authentication verification, CSRF safeguarding, and role-based authorization for administrative routes.
- **Apache Tomcat 9:** Embedded servlet container handling HTTP network requests, session lifecycles, and servlet mappings.
- **Apache Maven 3.8.6:** Project management and automated dependency build automation tool managing libraries and application packaging.
- **Git & GitHub:** Distributed version control system used for collaborative source management, branching, and codebase versioning.
- **Visual Studio Code / IntelliJ IDEA:** Integrated Development Environments (IDEs) utilized for enterprise Java programming, debugging, and terminal execution.

CHAPTER 3
SYSTEM DESIGN

3.1 DATABASE SCREENSHOTS

VARIOUS TABLES TO MAINTAIN INFORMATION

- **Customer Table from Database**

Stores user identities, credentials, contact addresses, and assigned security roles (ROLE_ADMIN vs ROLE_NORMAL).

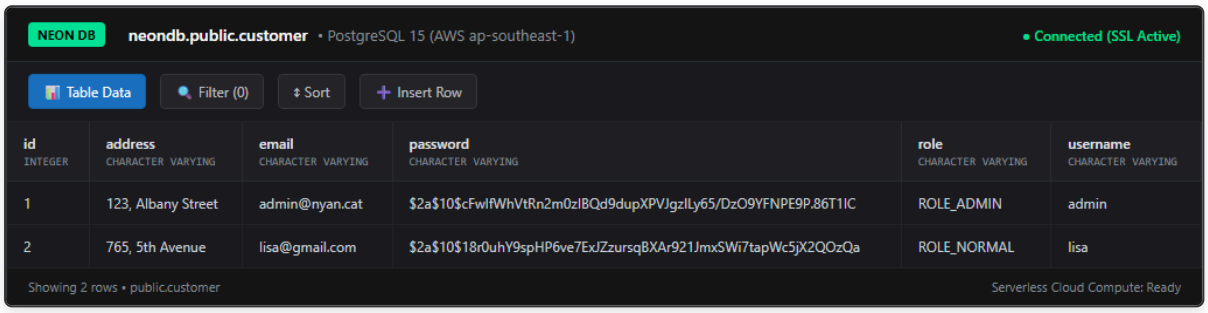
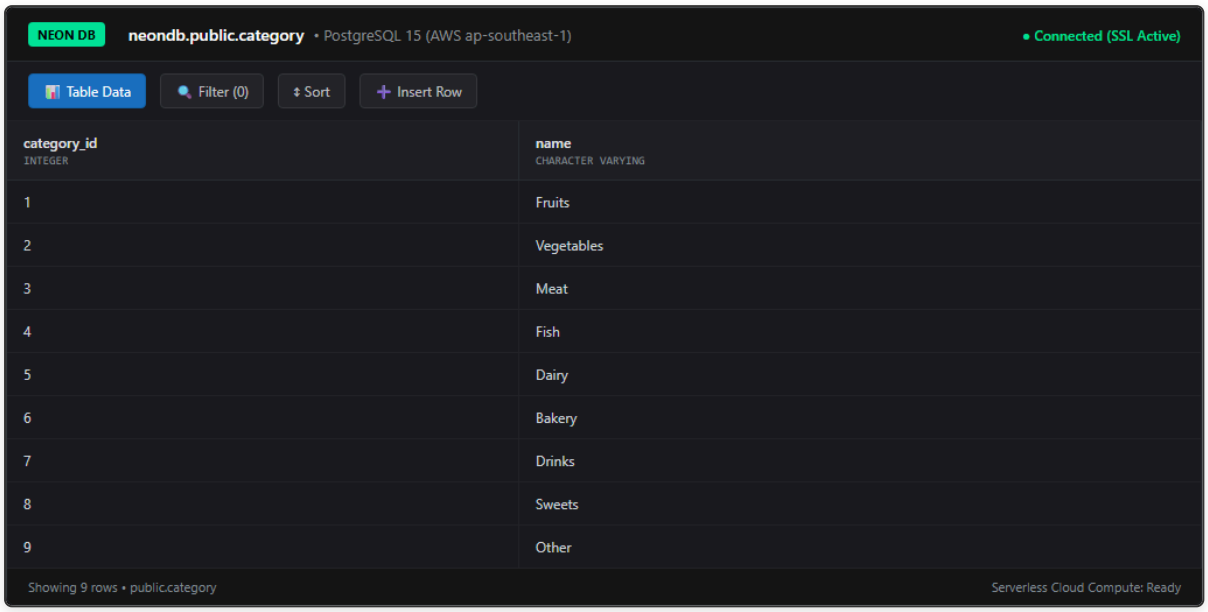


Figure 3.1: CUSTOMER Table in Neon Serverless PostgreSQL

- **Category Table from Database**

Maintains product classification categories (Fruits, Vegetables, Meat, Fish, Dairy, Bakery, Drinks, etc.).

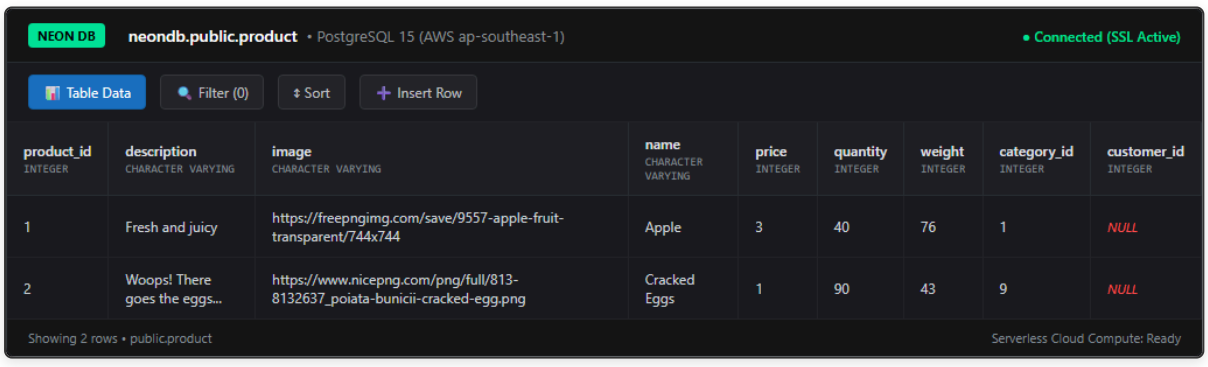


category_id INTEGER	name CHARACTER VARYING
1	Fruits
2	Vegetables
3	Meat
4	Fish
5	Dairy
6	Bakery
7	Drinks
8	Sweets
9	Other

Figure 3.2: CATEGORY Table in Neon PostgreSQL

- **Product Table from Database**

Stores commercial catalog items, prices, stock quantities, weight metrics, descriptions, image URLs, and category foreign keys.



product_id INTEGER	description CHARACTER VARYING	image CHARACTER VARYING	name CHARACTER VARYING	price INTEGER	quantity INTEGER	weight INTEGER	category_id INTEGER	customer_id INTEGER
1	Fresh and juicy	https://freepngimg.com/save/9557-apple-fruit-transparent/744x744	Apple	3	40	76	1	NULL
2	Woops! There goes the eggs...	https://www.nicepng.com/png/full/813-8132637_poiata-bunicii-cracked-egg.png	Cracked Eggs	1	90	43	9	NULL

Figure 3.3: PRODUCT Table in Neon PostgreSQL

- **Cart Table from Database**

Represents persistent customer shopping carts linked via foreign keys to registered consumer accounts.

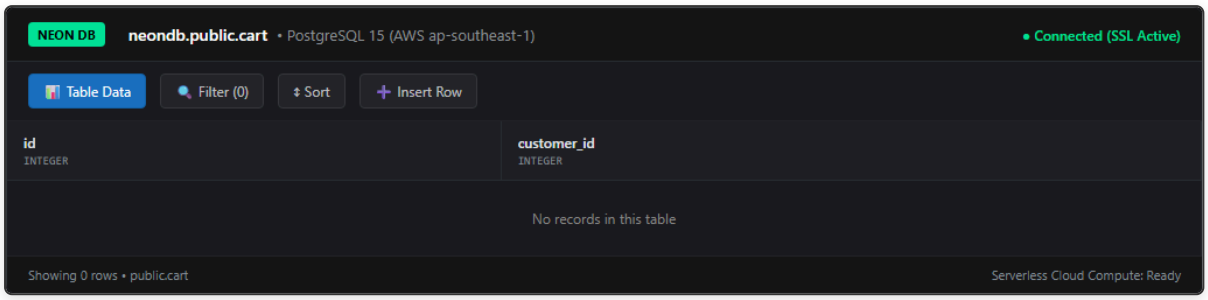


Figure 3.4: CART Table in Neon PostgreSQL

- **Cart_Product Table from Database**

Join table tracking composite primary keys (cart_id, product_id) representing items currently held in carts.

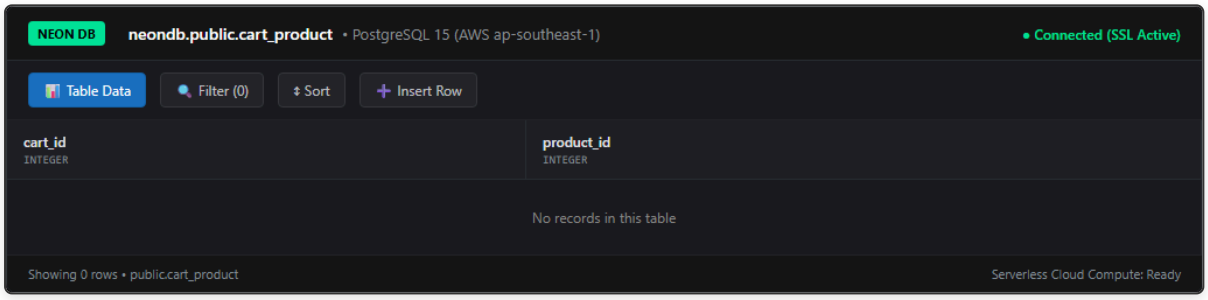


Figure 3.5: CART_PRODUCT Association Table in Neon PostgreSQL

3.2 DATA FLOW DIAGRAMS

- **DATA FLOW DIAGRAM FOR CUSTOMER**

Illustrates the customer interaction flow across authentication, catalog discovery, shopping cart aggregation, and cloud persistence.

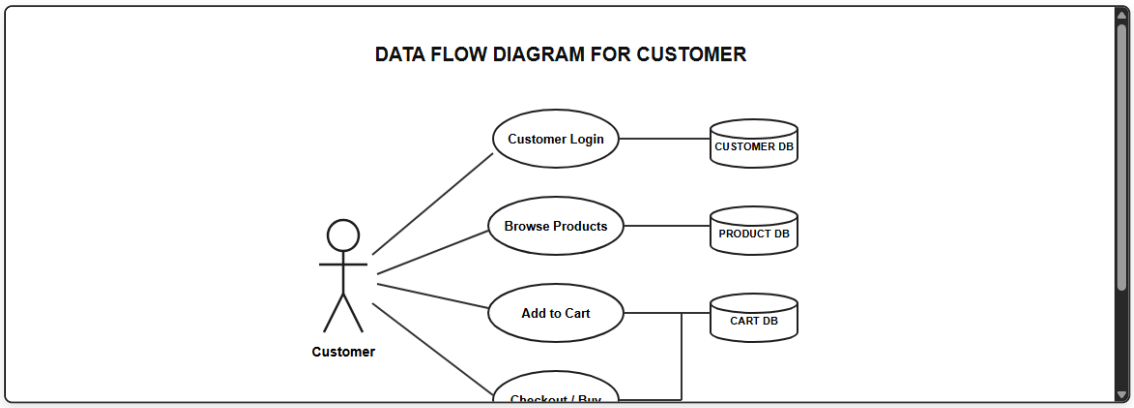


Figure 3.6: Data Flow Diagram (Level 1) for Customer Workflow

- **USE CASE DIAGRAM FOR USER & ADMIN**

Depicts the primary system actors (Customer and Administrator) and their respective interaction boundaries with system use cases.

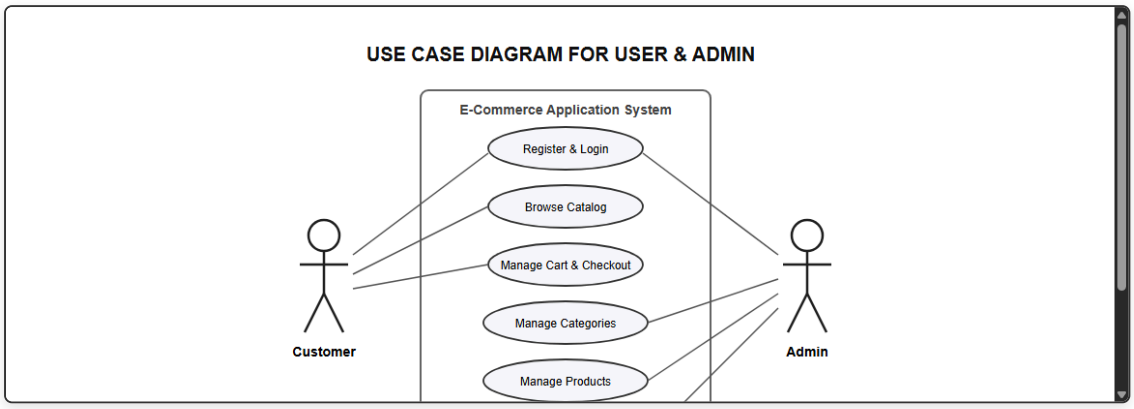


Figure 3.7: Use Case Diagram for Customer and Admin Workflows

CHAPTER 4
SYSTEM IMPLEMENTATION

4.1 SCREENSHOTS OF OUTPUT

4.1.1 Screenshot of Storefront / Home Page

The public-facing catalog page showcasing featured products fetched dynamically from Neon DB with direct "Buy Now" capabilities.

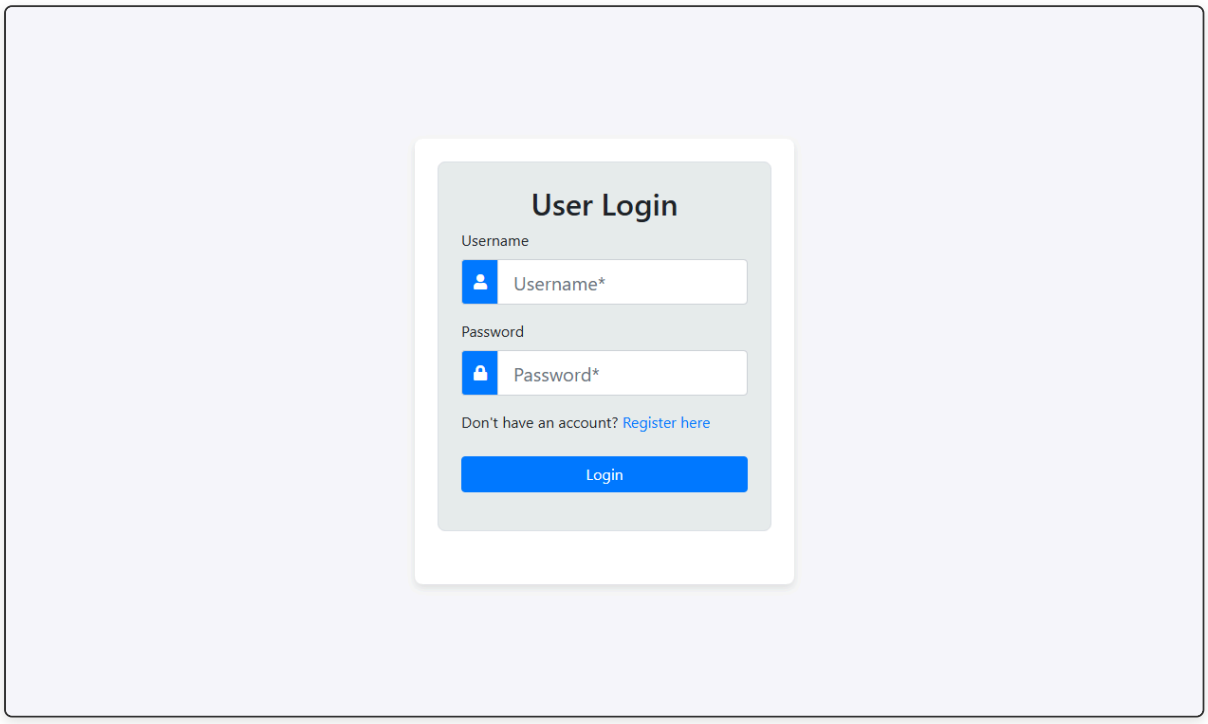


Figure 4.1: Customer Storefront / Home Page View

4.1.2 Screenshot of Customer Catalog & Order Checkout

Customer products browsing catalog interface and instant order confirmation / checkout page.

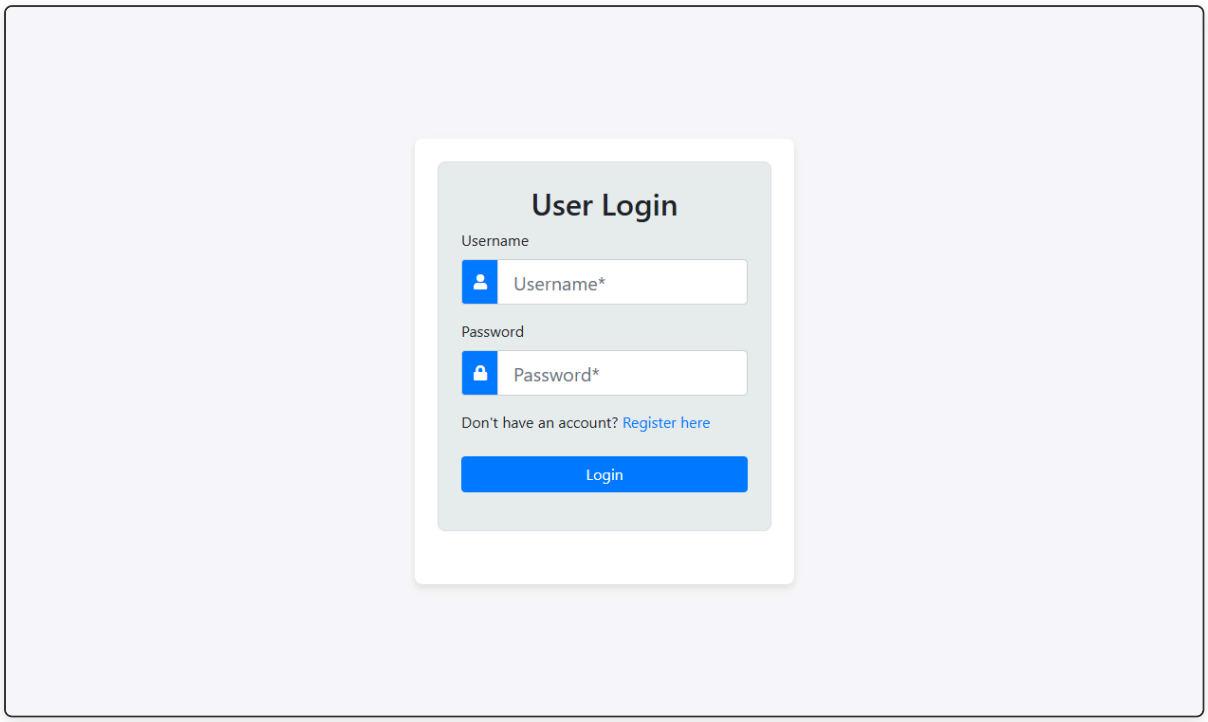


Figure 4.2: Customer Products Catalog Interface

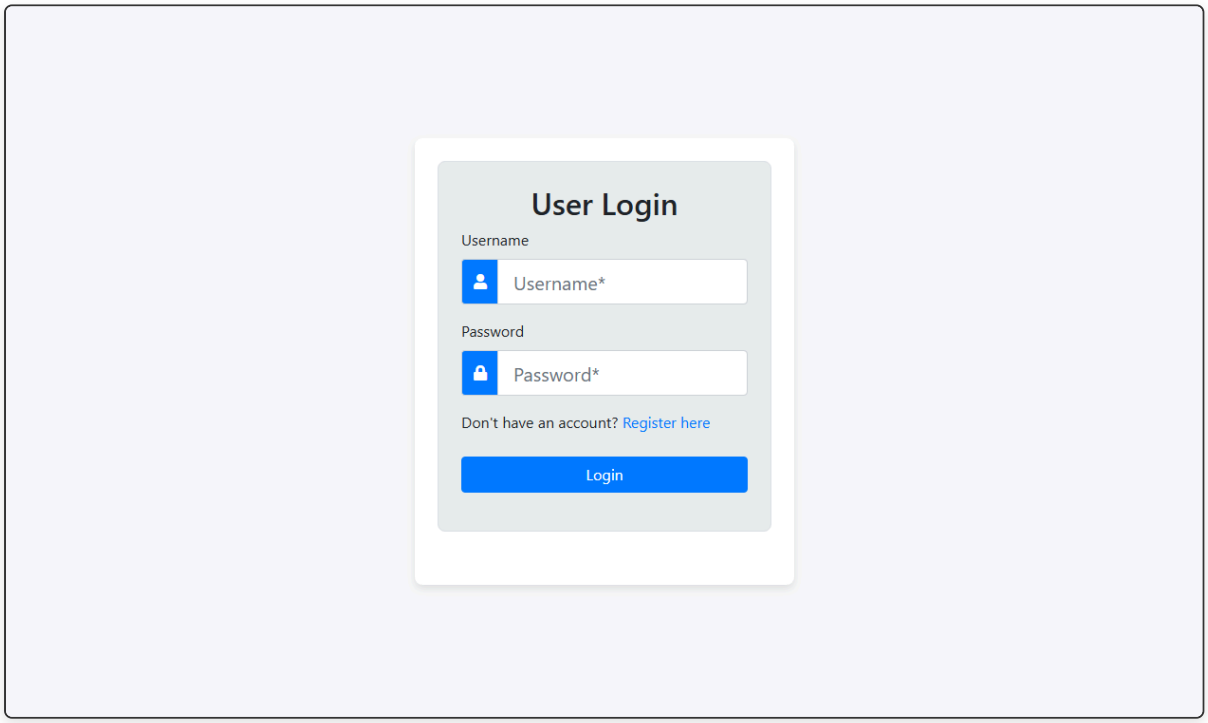


Figure 4.3: Customer Order Placement & Checkout Page

4.1.3 Screenshot of Customer Login & Registration

Secure portal interfaces allowing customer profile creation and Spring Security authenticated sessions.

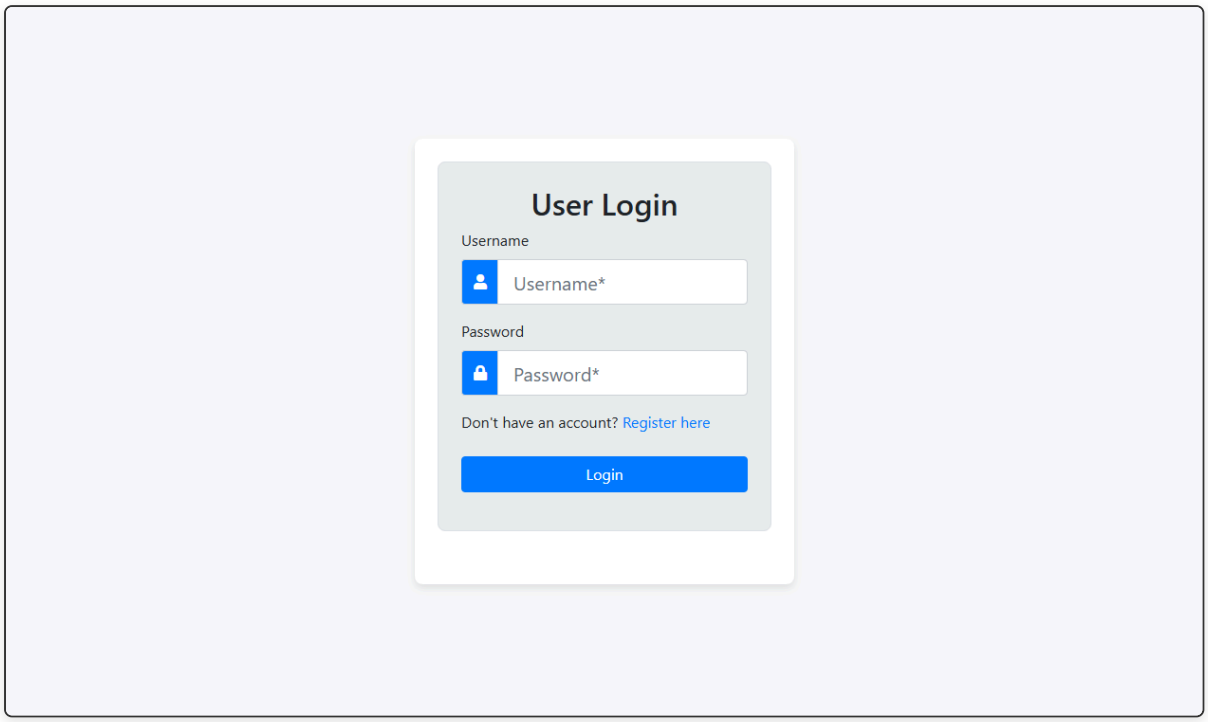


Figure 4.4: Customer Login Gateway

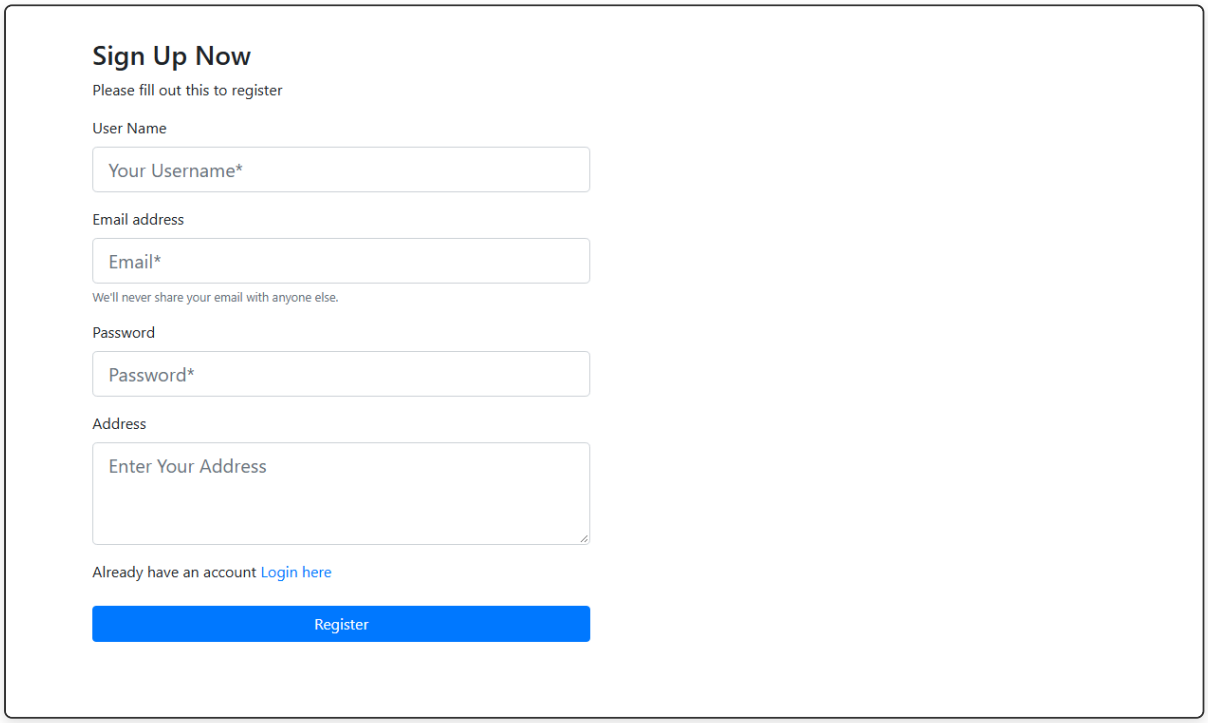


Figure 4.5: Customer Registration Form Interface

4.1.4 Screenshot of Customer Profile & Admin Login

Authenticated customer profile management settings and dedicated administrator portal login.

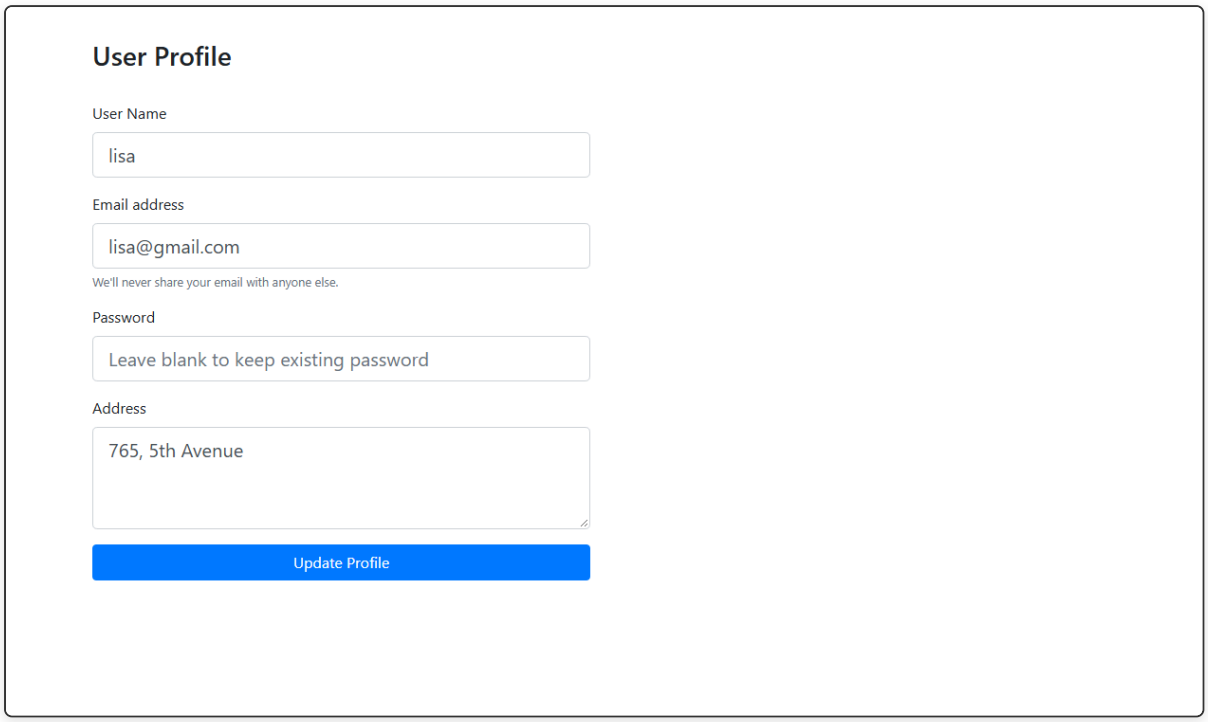


Figure 4.6: Customer Profile and Address Update Settings

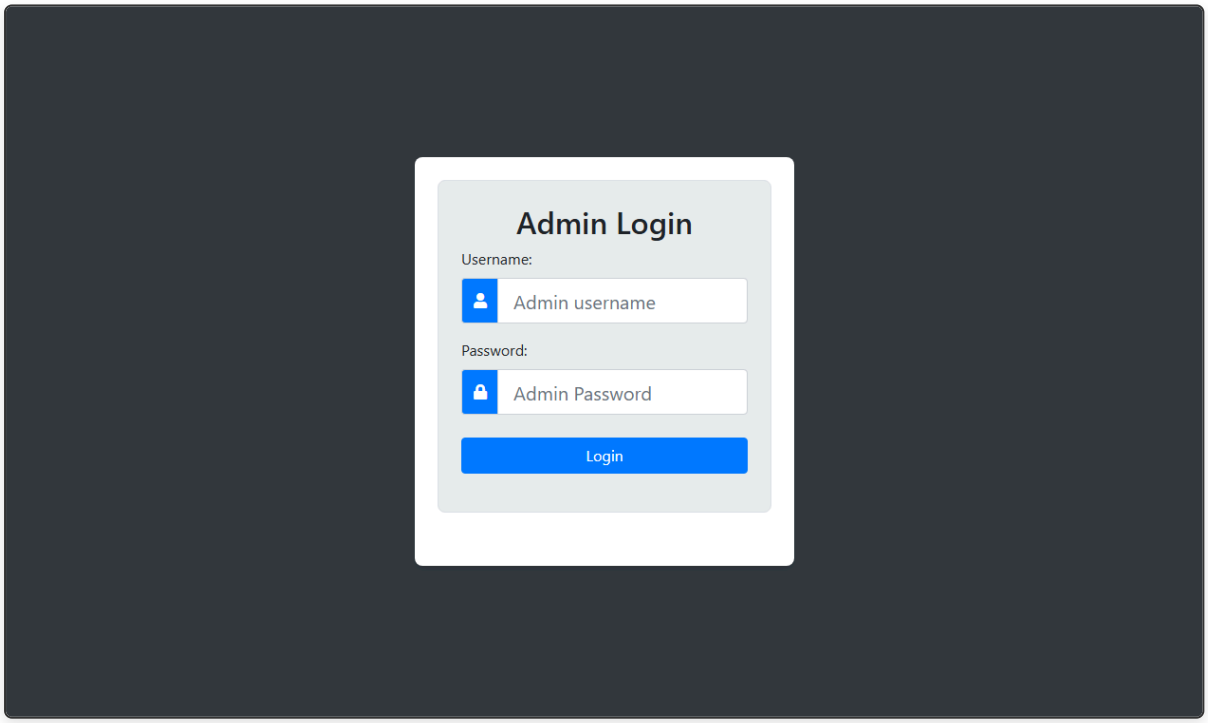


Figure 4.7: Dedicated Administrator Portal Login Interface

4.1.5 Screenshot of Administrator Dashboard

Central administrative mission control providing navigation across Category management, Product inventories, and Registered Customer directories.

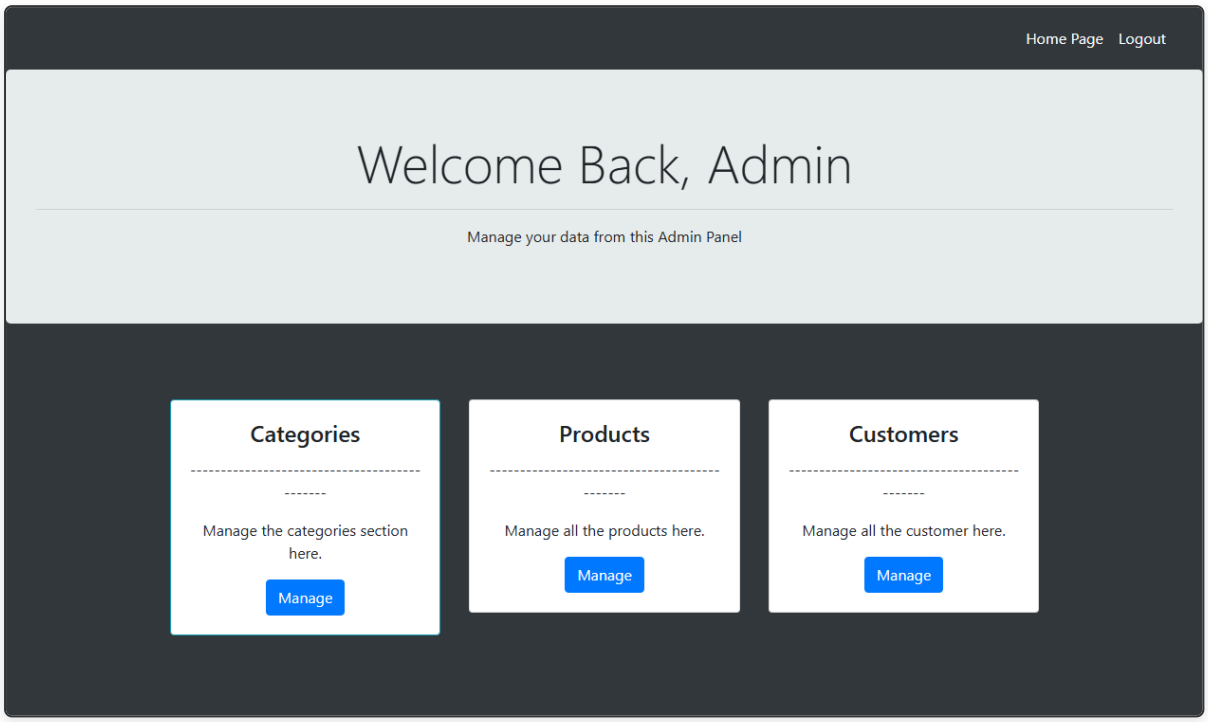


Figure 4.8: Central Administrator Control Dashboard

4.1.6 Screenshot of Category Management & Add Modal

Active categories taxonomy table and interactive "Add New Category" modal form dialog.

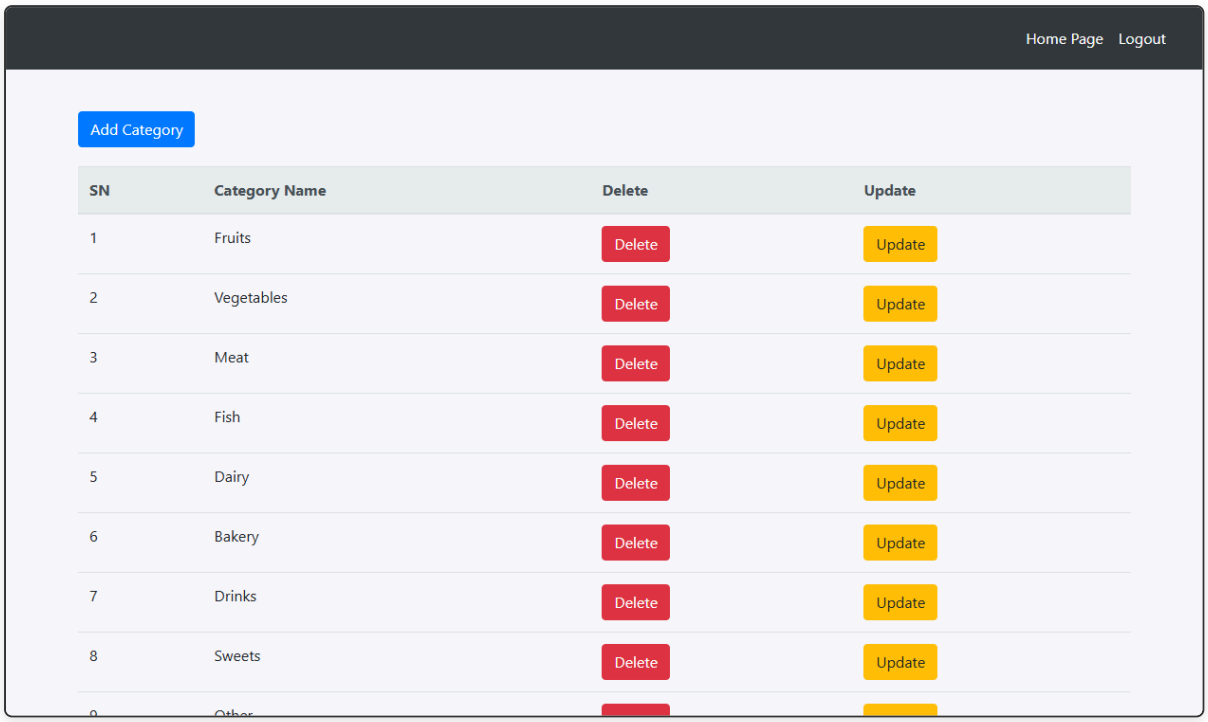


Figure 4.9: Category Management and Classification Table

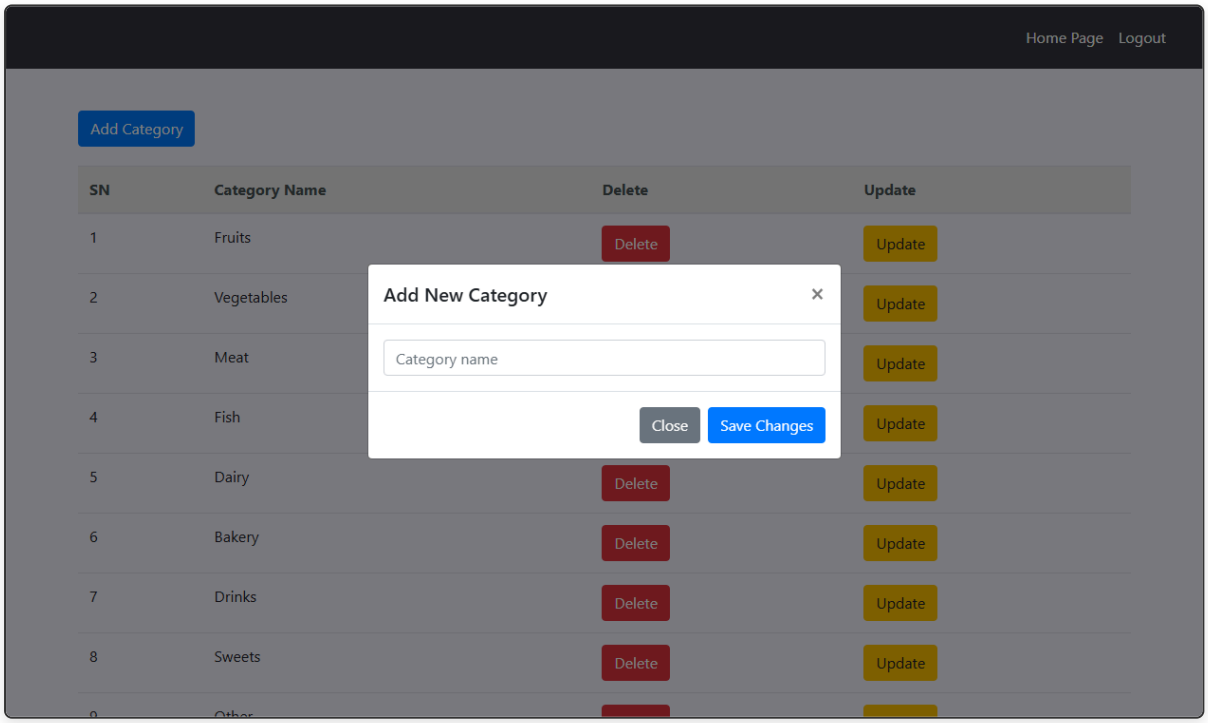


Figure 4.10: Add New Category Classification Modal Dialog

4.1.7 Screenshot of Product Inventory Overview

Product catalog overview displaying item images, product names, categories, price points, stock quantities, and action controls.

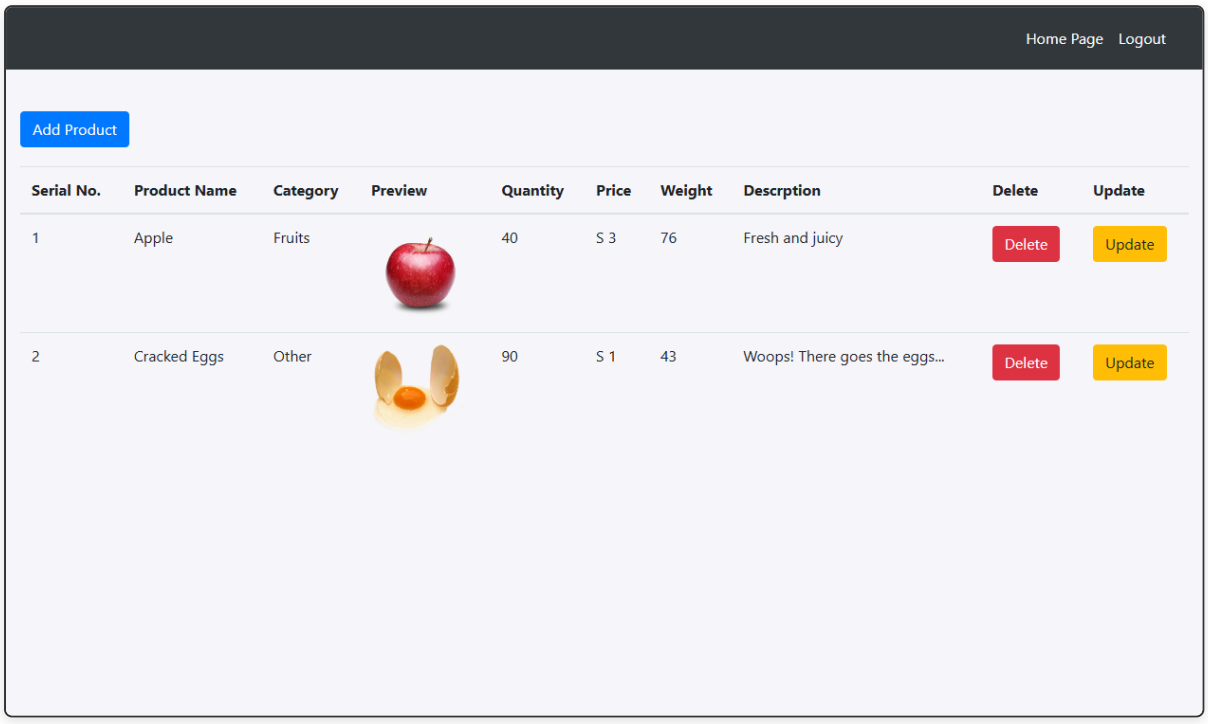


Figure 4.11: Inventory Product Management Overview Table

4.1.8 Screenshot of Product Ingestion & Update Forms

Full administrative forms used to add new products and update existing catalog entries in Neon DB.

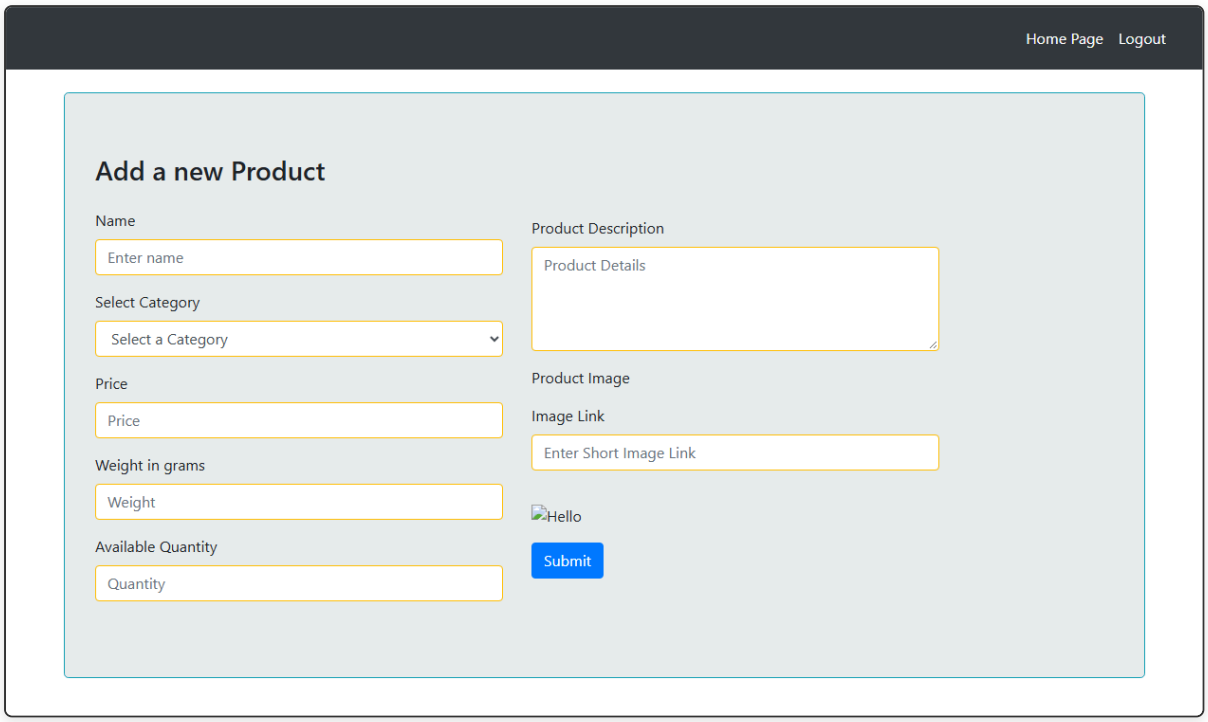


Figure 4.12: Add New Product Ingestion Form Interface

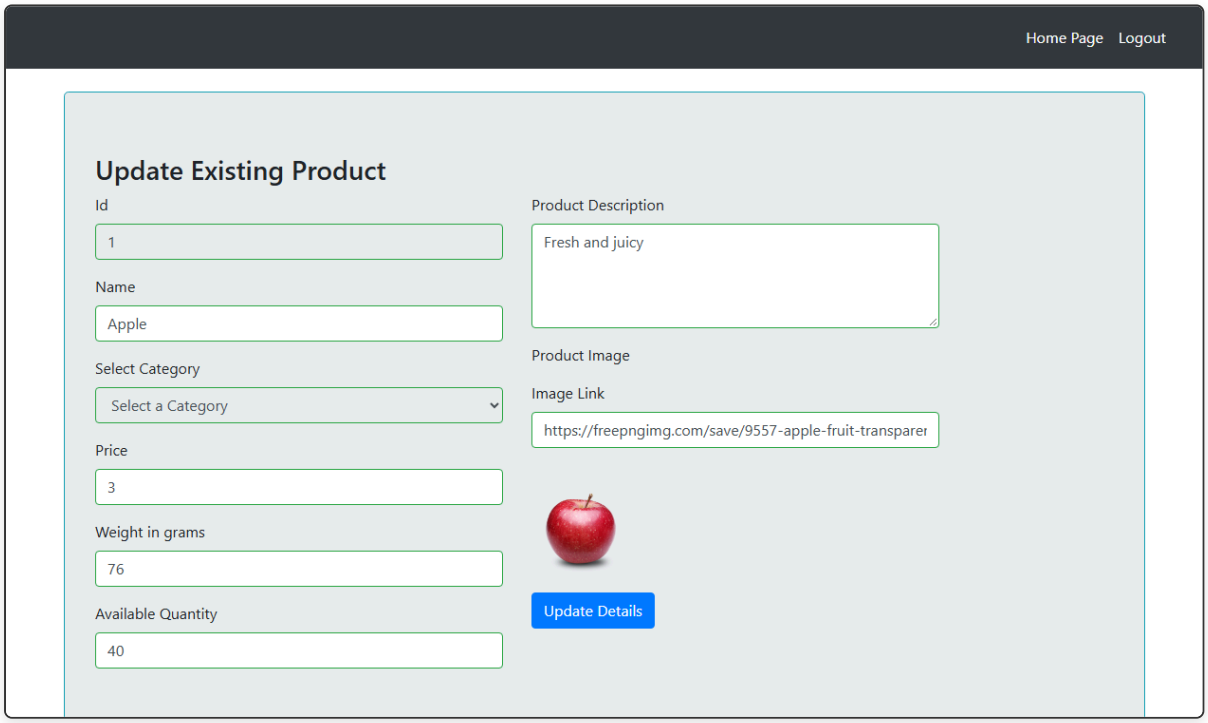


Figure 4.13: Edit / Update Product Information Form Interface

4.1.9 Screenshot of Customers Directory & Admin Profile

Registered customer auditing directory and administrator account profile settings.

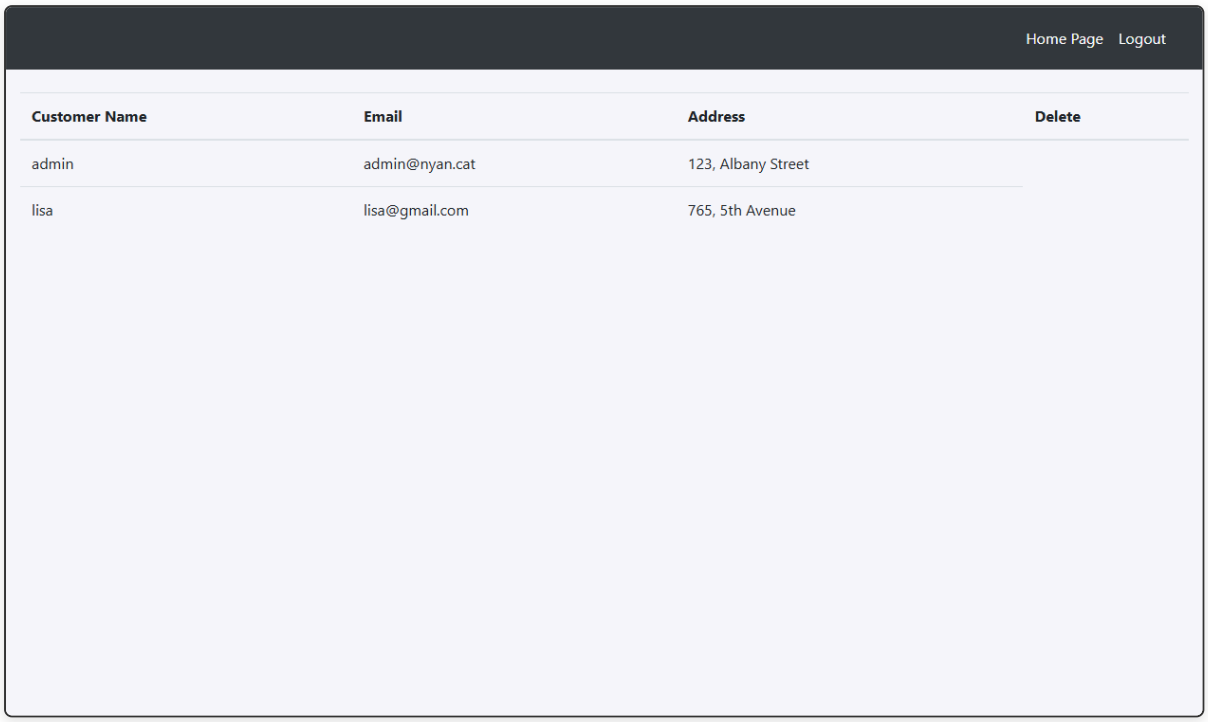


Figure 4.14: Registered Customer Auditing Directory Table

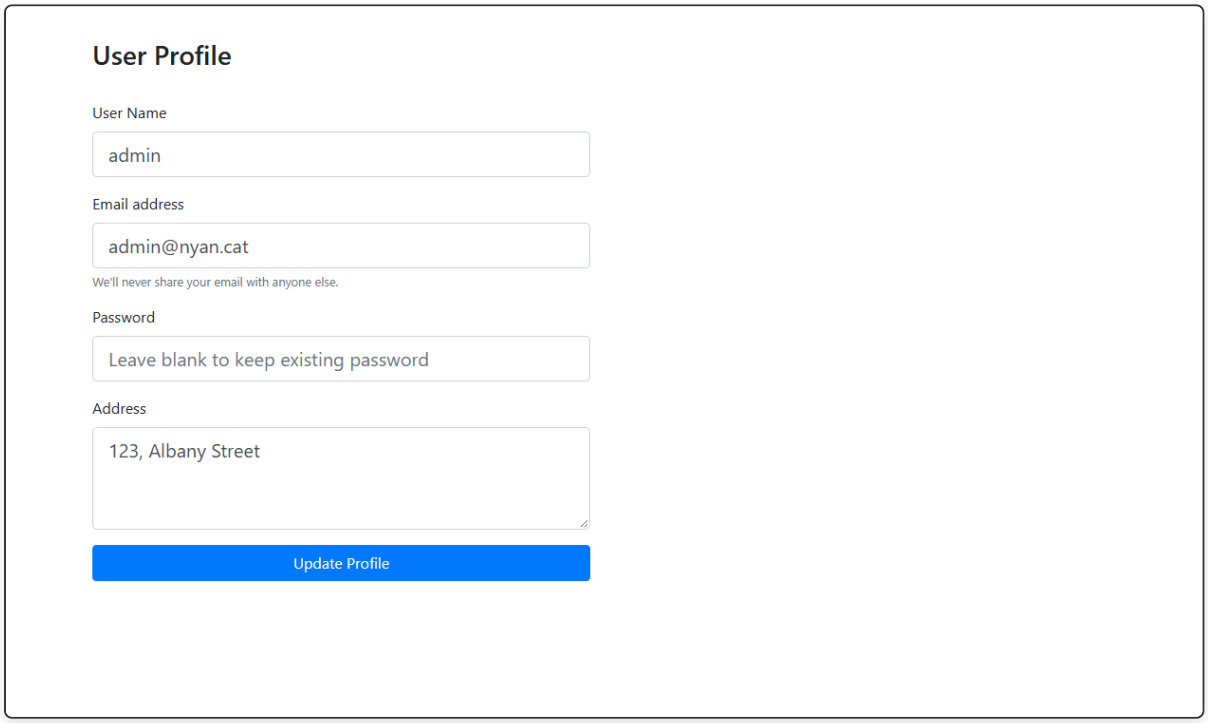


Figure 4.15: Administrator Account Profile Settings Interface

4.2 SCREENSHOTS OF CODING PART

- **Project Dependencies and Build Configuration (pom.xml)**

Maven project descriptor showing Spring Boot starters, JSP Jasper dependencies, and the PostgreSQL JDBC driver.

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
	xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
	xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
	<modelVersion>4.0.0</modelVersion>
	<parent>
		<groupId>org.springframework.boot</groupId>
		<artifactId>spring-boot-starter-parent</artifactId>
		<version>2.6.4</version>
		<relativePath /> <!-- lookup parent from repository -->
	</parent>
	<groupId>com.jtspringproject</groupId>
	<artifactId>JtSpringProject</artifactId>
	<version>0.0.1-SNAPSHOT</version>
	<name>JtSpringProject</name>
	<description>Spring project for Java Technology</description>
	<properties>
		<java.version>11</java.version>
	</properties>
	<dependencies>


		<dependency>
			<groupId>org.junit.jupiter</groupId>
			<artifactId>junit-jupiter</artifactId>
			<version>5.10.0</version>
			<scope>test</scope>
		</dependency>


		<dependency>
			<groupId>org.springframework.boot</groupId>
```

Figure 4.16: Maven Dependencies Configuration (pom.xml)

- **Database Connection & SessionFactory Configuration**

Hibernate configuration bean defining DriverManagerDataSource, LocalSessionFactoryBean, and transaction manager.

HibernateConfiguration.java

```java
package com.jtspringproject.JtSpringProject;

import java.util.Properties;

import javax.sql.DataSource;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.orm.hibernate5.HibernateTransactionManager;
import org.springframework.orm.hibernate5.LocalSessionFactoryBean;
import org.springframework.transaction.annotation.EnableTransactionManagement;

@Configuration
@EnableTransactionManagement
public class HibernateConfiguration {
    @Value("${db.driver}")
    private String DRIVER;

    @Value("${db.password}")
    private String PASSWORD;

    @Value("${db.url}")
    private String URL;

    @Value("${db.username}")
    private String USERNAME;

    @Value("${hibernate.dialect}")
    private String DIALECT;

    @Value("${hibernate.show_sql}")
    private String SHOW_SQL;

```

Figure 4.17: HibernateConfiguration.java Class

- **Application Configuration (application.properties)**

application.properties

```
# Server configuration
server.port=8080
spring.mvc.view.prefix=/views/
spring.mvc.view.suffix=.jsp

#Database related properties
#database.driver=com.mysql.cj.jdbc.Driver
#database.url=jdbc:mysql://localhost:3306/ecomjava
#database.user=root
#database.password=
#
##Hibernate related properties
#hibernate.dialect=org.hibernate.dialect.MySQL5Dialect
#hibernate.show_sql=true
#hibernate.format_sql=true
#hibernate.hbm2ddl.auto=update


# Hibernate
hibernate.dialect=org.hibernate.dialect.PostgreSQL10Dialect
hibernate.show_sql=true
hibernate.hbm2ddl.auto=update
#entitymanager.packagesToScan:

spring.jpa.properties.hibernate.enable_lazy_load_no_trans=true

```

- **Web Controllers Implementation**

Spring MVC controllers orchestrating customer request handling and administrative operations.

UserController.java

```java
@Controller
public class UserController {

	private final userService userService;
	private final productService productService;

	@Autowired
	public UserController(userService userService, productService productService) {
		this.userService = userService;
		this.productService = productService;
	}

	@GetMapping("/register")
	public String registerUser() {
		return "register";
	}

	@GetMapping("/buy")
	public String buy() {
		return "buy";
	}

	@GetMapping("/login")
	public ModelAndView userLogin(@RequestParam(required = false) String error) {
		ModelAndView mv = new ModelAndView("userLogin");
		if ("true".equals(error)) {
			mv.addObject("msg", "Please enter correct email and password");
		}
		return mv;
	}

	@GetMapping("/")
	public ModelAndView indexPage() {
```

Figure 4.19: UserController.java Controller Logic

AdminController.java

```java
@Controller
@RequestMapping("/admin")
public class AdminController {

	private final userService userService;
	private final categoryService categoryService;
	private final productService productService;
	private static final String REDIRECT_ADMIN_PRODUCTS = "redirect:/admin/products";
	private static final String REDIRECT_ADMIN_CATEGORIES = "redirect:/admin/categories";
	private static final String VIEW_CATEGORIES = "categories";

	@Autowired
	public AdminController(userService userService, categoryService categoryService, productService productService) {
		this.userService = userService;
		this.categoryService = categoryService;
		this.productService = productService;
	}

	@GetMapping("/index")
	public String index(Model model) {
		String username = SecurityContextHolder.getContext().getAuthentication().getName();
		model.addAttribute("username", username);
		return "index";
	}

	@GetMapping("login")
	public ModelAndView adminLogin(@RequestParam(required = false) String error) {
		ModelAndView mv = new ModelAndView("adminlogin");
		if ("true".equals(error)) {
			mv.addObject("msg", "Invalid username or password. Please try again.");
		}
		return mv;
	}
```

- **Entity Domain Models**

JPA-annotated domain entity models representing CUSTOMER and PRODUCT entities mapped to Neon PostgreSQL.

User.java

```java
package com.jtspringproject.JtSpringProject.models;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity(name = "CUSTOMER")
@Table
public class User {

	@Id
	@GeneratedValue(strategy = GenerationType.IDENTITY)
	private int id;

	@Column(unique = true)
	private String username;

	private String email;

	private String password;

	private String role;

	private String address;

	public int getId() {
		return id;
	}

```

Figure 4.21: User.java Domain Model

Product.java

```java
package com.jtspringproject.JtSpringProject.models;

import javax.persistence.CascadeType;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.ManyToOne;
import javax.persistence.OneToOne;

@Entity(name="PRODUCT")
public class Product {
	@Id
	@Column(name = "product_id")
	@GeneratedValue(strategy=GenerationType.AUTO)
	private int id;

	private String name;

	private String image;

	@OneToOne(cascade = CascadeType.ALL)
	@JoinColumn(name = "category_id",referencedColumnName = "category_id")
	private Category category;

	private int quantity;

	private int price;

	private int weight;
```

CHAPTER 5
SYSTEM TESTING

5.1 FUNCTIONALITY TESTING:

Functional testing is a fundamental aspect of software quality assurance, ensuring that every feature of the enterprise e-commerce application operates as intended. Below is an in-depth exploration of its components, types, and workflow:

What Is Functional Testing? Functional testing is a black-box testing technique that assesses a software system against its functional requirements. This approach focuses on what the system is designed to do, rather than how it achieves those outcomes, allowing testers to evaluate functionality without needing to understand the internal code structure.

Objectives of Functional Testing:

- Verify that each function of the software performs as expected (authentication, catalog browsing, cart operations, administrative CRUD).
- Ensure that user inputs yield the anticipated database modifications in Neon PostgreSQL.
- Confirm adherence to business logic, pricing computations, and role-based access restrictions.

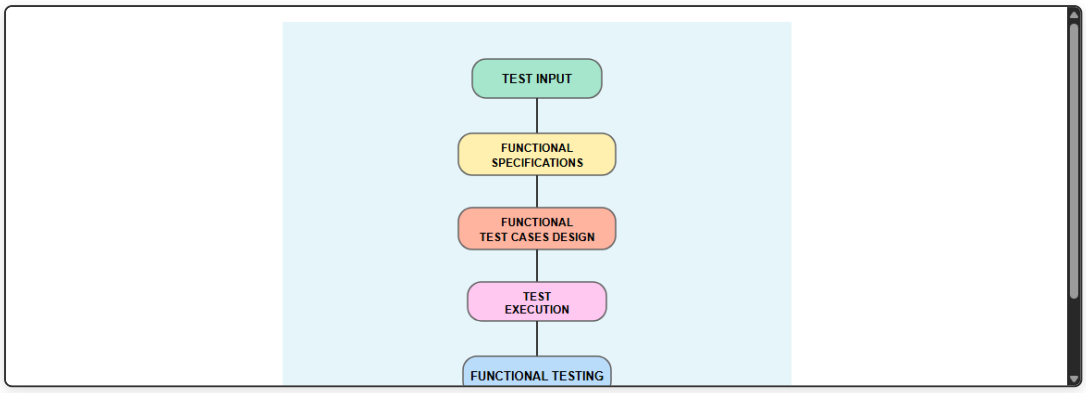


Fig: 5.1 Functional Testing Lifecycle Workflow

5.2 EXCEPTION TESTING WITHIN FUNCTIONAL TESTING :

While functional testing primarily evaluates whether software performs correctly under standard conditions, exception testing expands this scope by examining the system's behavior in error-prone or edge-case scenarios. This type of testing ensures that the system:

- Remains stable and does not crash when faced with invalid inputs, missing fields, or network failures.
- Effectively manages exceptions by displaying appropriate error messages and facilitating recovery.
- Preserves data integrity and security, even in adverse situations like cloud database disconnections.

Integrating Exception Testing into Functional Testing:

- **Identify Exception-Prone Functions:** Focus on modules that involve user input, external cloud database dependencies, or session authentication logic.
- **Design Exception Test Cases:** Include scenarios with invalid credentials, duplicate usernames, negative product quantities, and missing mandatory form fields.
- **Execute Tests with Monitoring:** Observe system behavior, Tomcat exception logs, and user-facing error banners during execution.
- **Verify Exception Handling:** Ensure that exceptions are caught, logged, and managed gracefully without disrupting server operation.
- **Report and Resolve:** Document any unhandled runtime exceptions or inadequate validation for prompt remediation.

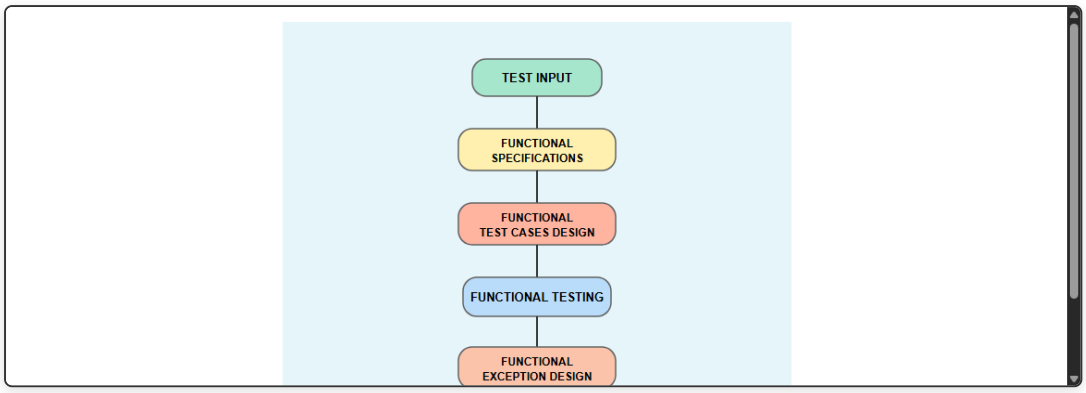


Fig: 5.2 Exception Testing Lifecycle Workflow

5.3 EXAMPLES

The matrix below outlines key functional testing and exception handling test cases executed across the e-commerce platform:

Functional Area	Exception / Scenario	Expected Behavior
Login Module	User enters wrong password repeatedly or nonexistent account	Display "Invalid username or password" banner; prevent unauthorized session creation
Registration Module	User attempts to register with already registered username	Trigger SQL unique constraint validation; inform user that username is already taken
Form Submission	Administrator attempts to create product with blank name or price	HTML5 form validation triggers; prompts user to fill missing mandatory fields
Role Security	Regular customer attempts direct URL access to /admin/products	Spring Security intercepts request; redirects to /admin/login due to missing ROLE_ADMIN
Database Connection	Transient cloud network interruption during checkout transaction	Hibernate catches SQLException; rolls back transaction to preserve financial integrity
Inventory Operations	Administrator deletes category linked to active inventory products	Foreign key integrity constraint enforced; prevents orphan records in PRODUCT table

CHAPTER 6
CONCLUSION

- The E-Commerce Enterprise Web Application project has successfully addressed the critical need for a scalable, secure, and easily maintainable digital shopping platform. Built upon the robust foundation of enterprise Java technologies—specifically Spring Boot 2.6.4, Hibernate ORM 5.6, and Neon Serverless PostgreSQL—the application delivers a high-performance commercial portal that seamlessly balances consumer convenience with administrative oversight.
- Throughout the development and implementation phases of this project, we successfully engineered key enterprise modules, including role-based security authentication, dynamic JSP storefront rendering, catalog categorization, inventory governance, and transactional shopping cart persistence.
- A pivotal architectural highlight of this project was the migration of the database tier from local storage to Neon's cloud-native serverless PostgreSQL platform. By utilizing cloud-hosted PostgreSQL with SSL encryption, the system achieves instant scalability, auto-provisioned connection pooling, and continuous high availability without requiring local server infrastructure.
- The user interface is thoughtfully engineered to be responsive and intuitive, ensuring that customers can effortlessly discover products and navigate checkout processes across all modern desktop and mobile web browsers.
- Looking forward, the project holds significant potential for future expansion, including integrating payment gateway APIs (Stripe, Razorpay), automated email notifications via Spring Mail, AI-driven recommendation engines, and containerized Docker deployments.

CHAPTER 7
REFERENCES

1. Spring Boot Reference Documentation — https://docs.spring.io/spring-boot/docs/2.6.4/reference/html/

2. Hibernate ORM 5.6 User Guide & SessionFactory Architecture — https://docs.jboss.org/hibernate/orm/5.6/userguide/html_single/Hibernate_User_Guide.html

3. Neon Serverless PostgreSQL Cloud Documentation — https://neon.tech/docs/introduction

4. PostgreSQL 15 Official Technical Manual & SQL Dialect Reference — https://www.postgresql.org/docs/15/index.html

5. Spring Security Architecture & Role-Based Access Control — https://spring.io/guides/topicals/spring-security-architecture

6. SRM Institute of Science and Technology, Department of Computing Technologies, Laboratory Manual for 21CSC203P Advanced Programming Practice, Academic Year 2025-2026.